12c Datenbankentwurf

Joern Ploennigs · AI4SC



> Bad programmers worry about the code. Good programmers worry about data structures and their relationships.
>
> - Linus Torvalds

(Gute Programmierer kuemmern sich um Datenstrukturen und ihre Beziehungen.)

Datenbankentwurf

Diese Sitzung liegt bewusst *vor* den SQL-Sitzungen. Die Idee ist dieselbe wie in `03c_Softwareentwurf`: Erst das Modell, dann die Implementierung. Im Datenbankkontext ist das Modell das ER-Diagramm. Es beschreibt, *welche* Informationen gespeichert werden, *wie* sie zusammenhaengen und *welche Regeln* gelten. Genau dieses Modell wird spaeter zur Spezifikation fuer AI-generierte `CREATE TABLE`-Statements.

Drei Ebenen des Datenbankentwurfs

Beim Datenbankentwurf arbeiten wir auf drei Ebenen. Zuerst kommt die *konzeptionelle Ebene*: Dort beschreiben wir die fachlichen Dinge der Realitaet, etwa Stationen, Sensoren und Messungen, ohne schon ueber SQL-Tabellen zu sprechen. Danach folgt die *logische Ebene*: Aus dem fachlichen Modell werden Tabellen, Primaer- und Fremdschluessel sowie Datentypen. Erst ganz am Ende betrachten wir die *physikalische Ebene*, also Indizes, Speicherstrukturen und Datenbanksystem-spezifische Optimierungen.

Das Umweltmonitoring-Beispiel zeigt gut, warum diese Trennung hilft: Konzeptionell fragen wir, welche Objekte und Beziehungen existieren. Logisch entscheiden wir, welche Tabellen und Fremdschluessel wir dafuer brauchen. Physikalisch waehlen wir spaeter aus, wie eine konkrete Datenbank diese Tabellen performant speichert.

ER-Diagramme als Informationsmodell

Ein ER-Diagramm beantwortet nicht die Frage, wie eine SQL-Syntax genau aussieht. Es beantwortet die wichtigere Vorfrage: *Welche Dinge der Realitaet muessen wir unterscheiden und wie haengen sie zusammen?* Genau deshalb passt die Sitzung inhaltlich zu `03c_Softwareentwurf`: Dort wurden Modelle als Denkwerkzeuge eingefuehrt, hier wird dieselbe Haltung auf Daten angewendet.

Die Grundidee laesst sich sofort am Umweltmonitoring zeigen. Eine Station ist ein fachliches Objekt, ein Sensor ist ein weiteres, und eine Messung dokumentiert ein Ereignis im Zeitverlauf. Das ER-Diagramm hilft uns, diese Objekte voneinander abzugrenzen, ihre Eigenschaften zu sammeln und ihre Beziehungen als Regeln zu formulieren, bevor wir an SQL denken.

ER-Modell - Grundbegriffe

Ein *Entitaetstyp* ist eine Klasse fachlicher Dinge, ueber die wir Informationen speichern wollen, zum Beispiel `Station`, `Sensor` oder `Measurement`. Eine *Entitaet* ist dann ein konkretes Exemplar eines solchen Typs, etwa die Station „Campus Nord“ oder ein bestimmter Temperatursensor. Ein *Attribut* beschreibt eine Eigenschaft einer Entitaet, zum Beispiel `standort`, `typ` oder `timestamp`.

Zwischen Entitaeten bestehen *Beziehungen* (Relationships). Im Monitoring-Beispiel gehoert ein Sensor zu genau einer Station, waehrend eine Station mehrere Sensoren haben kann. Genau diese Mengenangaben nennen wir *Kardinalitaeten*. Sie sind spaeter entscheidend dafuer, welche Fremdschluessel und welche Pflicht- oder Optionalregeln im relationalen Schema noetig werden.

UML-Notation fuer ER-Diagramme

In dieser Veranstaltung verwenden wir fuer ER-Diagramme eine UML-nahe Schreibweise. `<<Entity>>` markiert, dass ein Kasten keine Softwareklasse mit Methoden beschreibt, sondern einen Entitaetstyp der Datenwelt. Attribute werden aehnlich wie in Klassendiagrammen notiert, Primaerschluessel werden sichtbar markiert, und Beziehungen verbinden die Entitaeten mit Kardinalitaeten an den Enden.

Der Vorteil dieser Notation liegt in der Vertrautheit: Wer UML aus `04c_Objects` kennt, kann dieselbe visuelle Sprache nun fuer Datenmodelle verwenden. Der Unterschied ist aber wichtig: Im ER-Kontext interessieren uns keine Methoden und kein Laufzeitverhalten, sondern nur die Struktur der spaeter persistent zu speichernden Informationen.

Kardinalitaeten werden spaeter zu Constraints

Kardinalitaeten sind die Bruecke zwischen konzeptionellem Modell und relationaler Umsetzung. Sie sagen nicht nur aus, *dass* zwei Entitaetstypen verbunden sind, sondern auch, wie viele Exemplare auf beiden Seiten zulaessig oder erforderlich sind. Wichtig ist dabei: Jede Beziehung hat immer zwei Blickrichtungen. „Eine Station hat `1..*` Sensoren“ und „ein Sensor gehoert zu genau `1` Station“ beschreiben dieselbe Beziehung, aber aus zwei unterschiedlichen Richtungen.

Diese Aussagen werden spaeter zu konkreten SQL-Folgen. Gehoert ein Sensor zwingend zu genau einer Station, braucht die Tabelle `Sensor` einen Fremdschluessel `station_id`, der `NOT NULL` sein muss. Darf eine Entitaet auf der Gegenseite fehlen, ist `NULL` moeglich. Erlaubt die Kardinalitaet

viele zugeordnete Datensaetze, entstehen mehrere Zeilen auf der referenzierenden Seite oder gegebenenfalls eine eigene Zwischentabelle.

Beispielkontext: Umweltmonitoring

Statt eines abstrakten Lehrbuchbeispiels betrachten wir ein typisches Ingenieurproblem: Messstationen im Feld, Sensoren an jeder Station und laufend eintreffende Messungen. Dieses Beispiel ist nah an Umwelt- und Bauingenieurpraxis und zeigt direkt, warum sauberes Datenmodellieren vor SQL notwendig ist.

Beispiel: Entitaeten und Attribute

Aus einer fachlichen Beschreibung lassen sich Entitaetstypen meist ueber die Dinge ableiten, ueber die dauerhaft Informationen gespeichert werden sollen. Im Umweltmonitoring sind das zunaechst `Station`, `Sensor` und `Measurement`. Zu jeder Station wollen wir etwa Namen und Standort kennen, zu jedem Sensor Typ und Hoehe und zu jeder Messung Zeitpunkt, Wert und Einheit.

Die Frage „Ist etwas ein eigener Entitaetstyp oder nur ein Attribut?“ ist zentral. `standort` ist nur eine Eigenschaft der Station und braucht deshalb keinen eigenen Kasten. `Sensor` dagegen ist ein eigenstaendiges Ding mit eigener Identitaet und eigenen Eigenschaften. Solche Entscheidungen sind der Kern des konzeptionellen Datenbankentwurfs.

Beispiel: Beziehungen und Kardinalitaeten

Beziehungen erkennt man in natuerlicher Sprache oft an Verben wie „hat“, „gehoert zu“ oder „produziert“. Im Monitoring-Beispiel hat eine Station mehrere Sensoren, und ein Sensor produziert viele Messungen. Aus diesen Aussagen entstehen nicht nur Verbindungslinien im Diagramm, sondern auch Kardinalitaeten an beiden Enden.

Die Formulierung in beide Richtungen ist didaktisch und fachlich wichtig. „Eine Station hat `1..*` Sensoren“ bedeutet zugleich: „Ein Sensor gehoert zu genau `1` Station.“ Ebenso gilt: „Ein Sensor produziert `0..*` Messungen“ und „eine Messung gehoert zu genau `1` Sensor.“ Diese doppelte Sicht hilft spaeter beim Pruefen, ob ein Fremdschluessel optional oder verpflichtend sein muss.

Beispiel: Textuelles ER-Diagramm

Ein ER-Modell muss nicht zwingend als Grafik vorliegen, um nuetzlich zu sein. Schon eine praezise Textform kann ein belastbares Entwurfsartefakt sein, solange Entitaetstypen, Attribute, Schluessel und Beziehungen eindeutig beschrieben sind. Gerade fuer agentische Workflows ist das hilfreich, weil ein Sprachmodell mit einer klaren Textspezifikation oft direkt weiterarbeiten kann.

Fuer den Unterricht ist die Textform ausserdem ein gutes Zwischengeruest: Sie zwingt dazu, alle Modellentscheidungen explizit auszuformulieren. Wer einen fachlichen Zusammenhang nicht in Text beschreiben kann, hat ihn meist auch im Diagramm noch nicht wirklich verstanden.

Vom ER-Modell zur Tabelle

Beim Uebergang zum logischen Schema wird jeder Entitaetstyp typischerweise zu einer Tabelle. Die Attribute werden zu Spalten, und der markierte Primaerschluessel wird zum `PRIMARY KEY`

der Tabelle. Beziehungen werden meist dadurch umgesetzt, dass auf der passenden Seite ein Fremdschluessel eingefuegt wird.

Welche Seite den Fremdschluessel traegt, haengt von der Kardinalitaet ab. Wenn ein Sensor zu genau einer Station gehoert, traegt `Sensor` den Fremdschluessel `station_id`. Wenn eine Messung zu genau einem Sensor gehoert, traegt `Measurement` den Fremdschluessel `sensor_id`. Genau hier wird sichtbar, warum ein gutes ER-Modell die eigentliche Spezifikation ist: Das SQL darf die fachliche Aussage nur technisch ausdruecken, nicht eigenmaechtig veraendern.

Normalisierung

Der konzeptionelle Entwurf ist noch nicht automatisch ein gutes relationales Schema. Beim Uebergang zum logischen Entwurf werden Redundanzen reduziert und nicht-atomare Strukturen aufgeloest. Genau das meint Normalisierung.

Normalisierung ist dabei kein Selbstzweck. Sie soll verhindern, dass dieselbe Information an vielen Stellen wiederholt wird oder dass mehrere Werte in einer einzigen Tabellenzelle versteckt werden. Im Monitoring-Beispiel waere es unguenstig, Sensorlisten als JSON in einer Stations-Tabelle zu speichern oder den Stationsnamen in jeder einzelnen Messzeile mitzuschleppen.

Warum wir normalisieren

Relationale Tabellen erwarten atomare Werte: In jede Zelle gehoert genau ein klar interpretierbarer Wert. Sobald wir Listen, eingebettete Strukturen oder mehrfach wiederholte Informationen in einer Tabelle sammeln, entstehen Pflegeprobleme. Eine Aenderung am Stationsnamen muesste dann vielleicht in tausenden Messzeilen nachvollzogen werden, und widerspruechliche Werte waeren fast unvermeidlich.

Im Monitoring-Beispiel fuehrt Normalisierung deshalb zu einer feineren logischen Struktur als das alltagssprachliche Fachmodell. Zwar sprechen wir fachlich locker ueber „eine Station mit Sensoren und Messungen“, aber relational zerlegen wir diese Welt in mehrere Tabellen, damit jede Information genau dort lebt, wo sie hingehoert.

Normalformen in einem Satz

Die ersten drei Normalformen reichen fuer einen ersten Entwurfsblick meist aus. *1NF* fordert atomare Werte ohne Listen oder verschachtelte Strukturen. *2NF* verlangt bei zusammengesetzten Schluesseln, dass Nichtschluesselattribute vom ganzen Schluessel abhaengen und nicht nur von einem Teil. *3NF* verbietet schliesslich transitive Abhaengigkeiten, also Redundanzen, bei denen ein Nichtschluesselattribut indirekt ein anderes bestimmt.

Im Monitoring-Beispiel ist 2NF oft schnell erfuellt, weil die Tabellen einfache technische IDs als Primaerschluessel tragen. 3NF wird dagegen relevant, sobald man versucht, Stations- oder Sensoreigenschaften in die Mess-Tabelle zu kopieren. Dann haengen diese Daten nicht direkt von der Messung selbst ab, sondern eigentlich vom referenzierten Sensor oder von der referenzierten Station.

1NF im Monitoring-Beispiel

Die erste Normalform verbietet mehrwertige oder komplex verschachtelte Attribute in einer einzelnen Tabellenzelle. Eine Spalte `sensoren JSON` in der Tabelle `Station` waere deshalb fuer ein relationales Schema problematisch, weil in einer Zelle ploetzlich viele Sensorobjekte versteckt waeren. Solche Daten lassen sich schwer pruefen, schwer abfragen und nur unsauber mit Fremdschluesseln absichern.

Die bessere Loesung ist eine eigene Tabelle `Sensor`, in der jeder Sensor eine eigene Zeile bekommt und ueber `station_id` mit seiner Station verbunden wird. So bleibt jeder Wert atomar, und die Beziehung wird offen und kontrollierbar im Schema sichtbar.

3NF im Monitoring-Beispiel

Die dritte Normalform verhindert, dass Informationen mehrfach und indirekt gespeichert werden. Wenn in jeder Messzeile neben `sensor_id` auch noch `sensor_typ` und `station_name` stehen, wiederholen sich diese Angaben fuer jede einzelne Messung. Aendert sich spaeter der Stationsname oder wird ein Sensortyp korrigiert, muessen sehr viele Zeilen angepasst werden.

Ein sauber normalisiertes Schema trennt deshalb `Station`, `Sensor` und `Measurement`. Die Messung verweist nur auf den Sensor, und der Sensor verweist auf die Station. So liegt jede Information genau an einer Stelle, und Inkonsistenzen werden deutlich unwahrscheinlicher.

ER-Diagramm zu Prompt

Hier wird die neue agentische Perspektive explizit: Das ER-Modell ist nicht nur ein Entwurfsdokument, sondern direkt die Grundlage fuer den Prompt, mit dem AI die DDL generieren soll.

Quiz

Referenzen

Bibliographie